OpenStack とAOSP プロジェクトを対象に 逆強化学習を用いたモジュール別レビュー動態予測

和歌山大学
◯ 橋本一輝，伊原彰紀

wakayama univ.

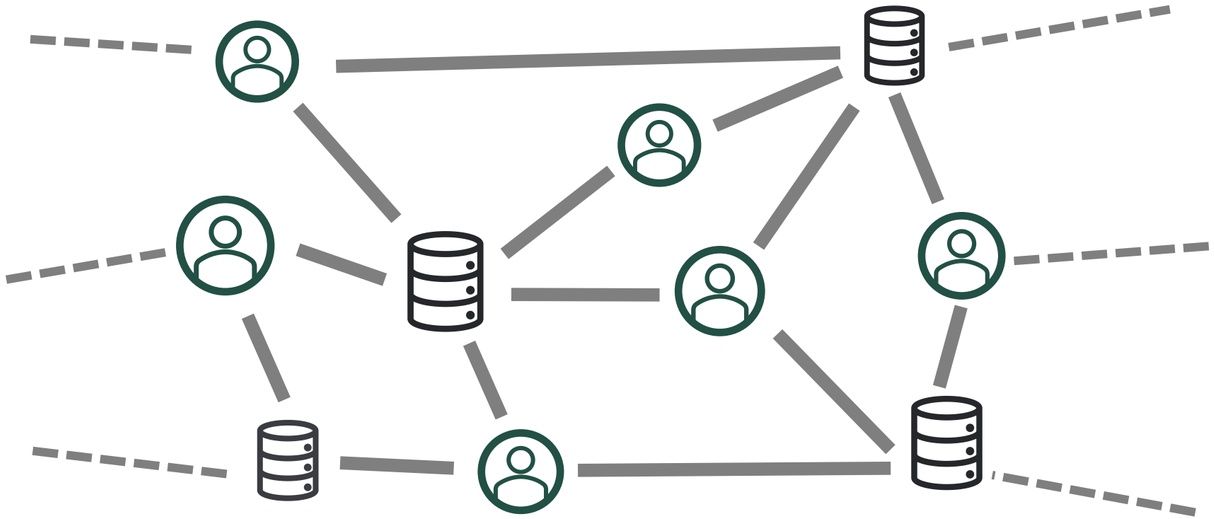
背景：オープンソースソフトウェア(OSS)開発の形態

地理的に分散した開発者が自主的に参加するバザール方式[1]

- 多様なボランティア開発者の貢献により成り立つ

開発者の多様な貢献の動機

- 知的探究心，技術向上，実績など貢献の動機が開発者による異なる[2]



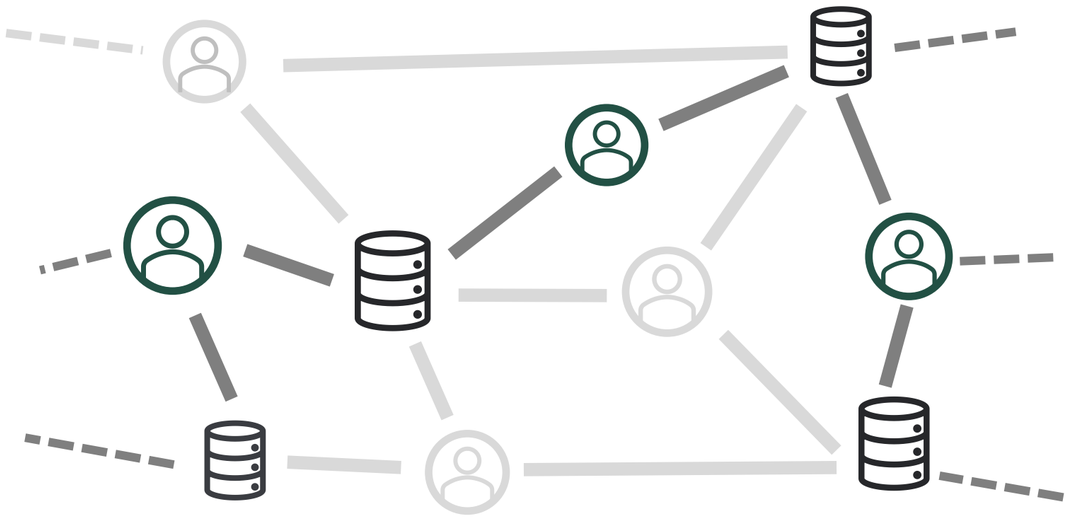
[1] Raymond, E.: The cathedral and the bazaar, Knowledge, Technology & Policy,
[2] Toward an understanding of the motivation of open source software developers, 25th International Conference on Software Engineering, 2003. Proceedings.

ボランティア開発者が主体であるバザール方式のメリット

メリット

- 多様な専門知識を持つ開発者が集結
- 24時間開発が可能

地域の時差を活用することでどこかで開発が進む



デメリット

- 貢献するタイミングや量が不安定
- 開発者の流動性が高い
  - 多くの開発者は一度の貢献で離脱

開発者の離脱により保守運用ができなくなる

[3] Lee, A. and Carver, J. C.: Are One-Time Contributors Different? A Comparison to Core and Periphery Developers in FLOSS Repositories, 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)

ボランティア開発者が主体であるバザール方式のデメリット

メリット

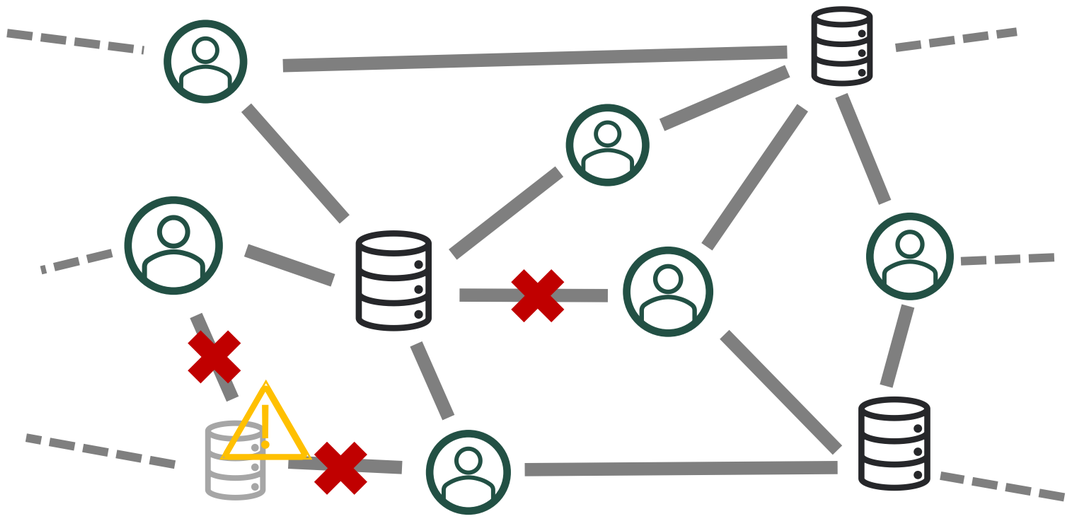
- 多様な専門知識を持つ開発者が集結
- 24時間開発が可能

地域の時差を活用することでどこかで開発が進む

デメリット

- 貢献するタイミングや量が見積りにくい
- 開発者の流動性が高い
  - 多くの開発者は一度の貢献で離脱[3]

開発者の離脱により保守運用ができなくなる



[3] Lee, A. and Carver, J. C.: Are One-Time Contributors Different? A Comparison to Core and Periphery Developers in FLOSS Repositories, 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)

プロジェクトの持続可能性について

開発リソースの確保と見積もり

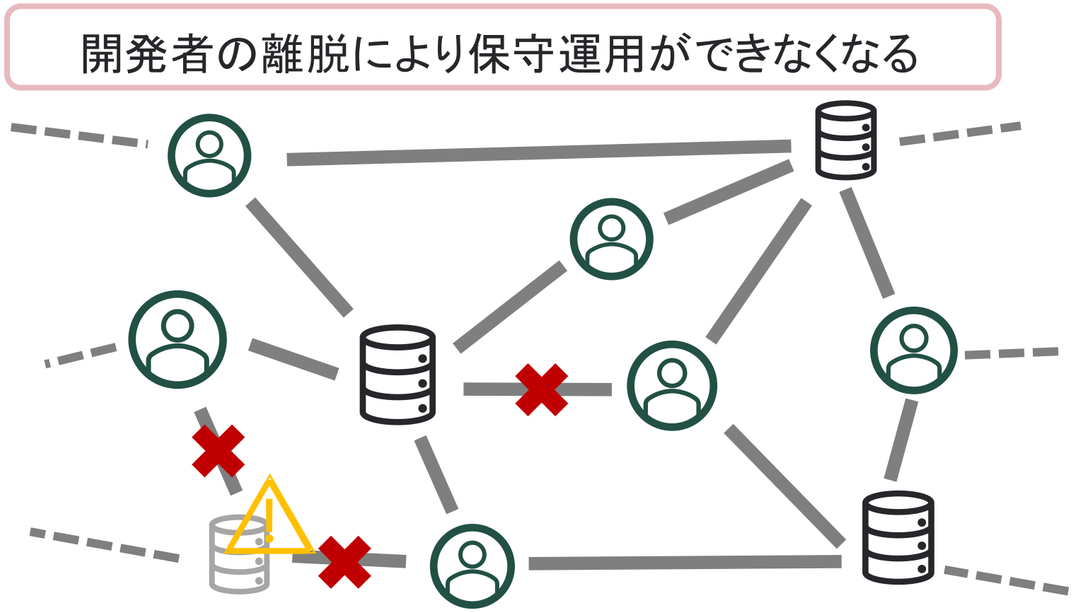
- 長期的に貢献する開発者が重要
- 安定した保守・運用のために事前に労働力の見積もりが必要

コア開発者の特定とサポート

- 単なる労働力の減少に留まらず，専門知識の喪失を招く

デメリット

- 貢献するタイミングや量が見積りにくい
- 開発者の流動性が高い
  - 多くの開発者は一度の貢献で離脱[3]



[3] Lee, A. and Carver, J. C.: Are One-Time Contributors Different? A Comparison to Core and Periphery Developers in FLOSS Repositories, 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)

関連研究

開発者の継続・離脱の要因分析

- 個人要因（私生活の出来事・興味の喪失）と組織要因（貢献者間の対立・方針の変化）により活動を停止[1]
  - 多様な理由で継続・停止することが示され，事前に継続性を予測する手法が求められる

継続予測：単一時点以降の貢献の有無を予測

- ZhouとMockus[2]：参加後 1 か月の累積活動量から、1 年後以降も貢献するかを予測
- Baoら[3]：GitHub の開発者が1年後や3年後に同じプロジェクトへ貢献するかを予測
- Ruangwanら[4]：ある特定時点のレビュー依頼に対して、開発者が貢献するか否かを予測

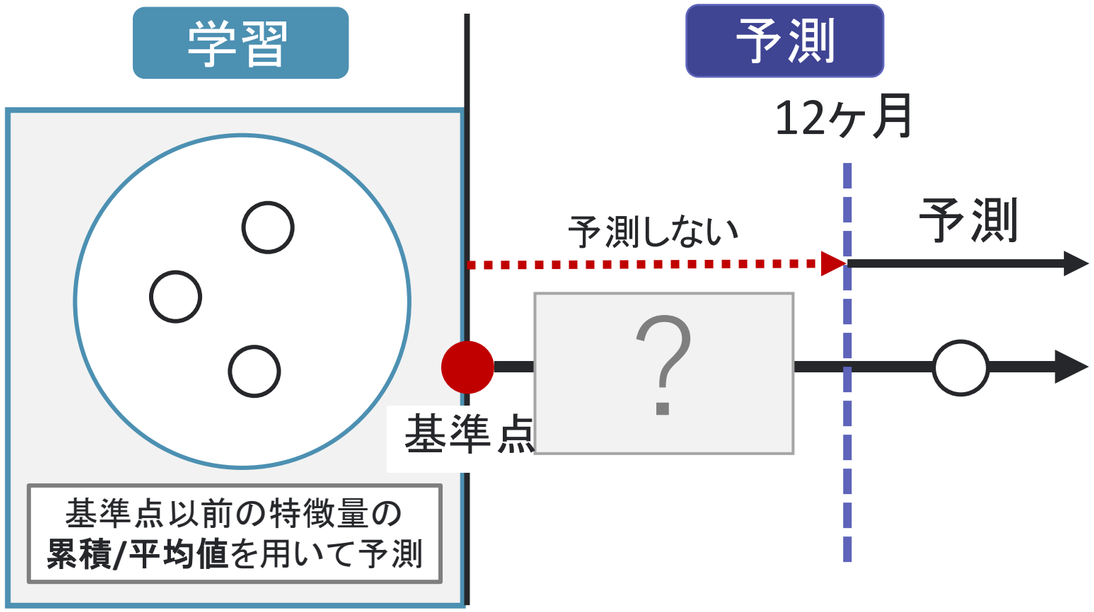
いずれも累積統計量を説明変数とした「特定時点以降に貢献するか」を予測する研究
⇒ 開発者の離脱・復帰や担当モジュールの時間的な移り変わりは捉えられない

[1] Iaffaldano, G. et al.: Why do developers take breaks from contributing to OSS projects? A preliminary analysis, arXiv:1903.09528 (2019). [2] Zhou, M. and Mockus, A.: What make long term contributors: Willingness and opportunity in OSS community, ICSE, pp. 518–528 (2012).
[3] Bao, L. et al.: A large scale study of long-time contributor prediction for GitHub projects, IEEE TSE, Vol. 47, No. 6, pp. 1277–1298 (2019). [4] Ruangwan, S. et al.: The impact of human factors on the participation decision of reviewers in modern code review, EMSE, Vol. 24, No. 2, pp. 973–1016 (2019).

従来研究と本研究の違い

従来研究

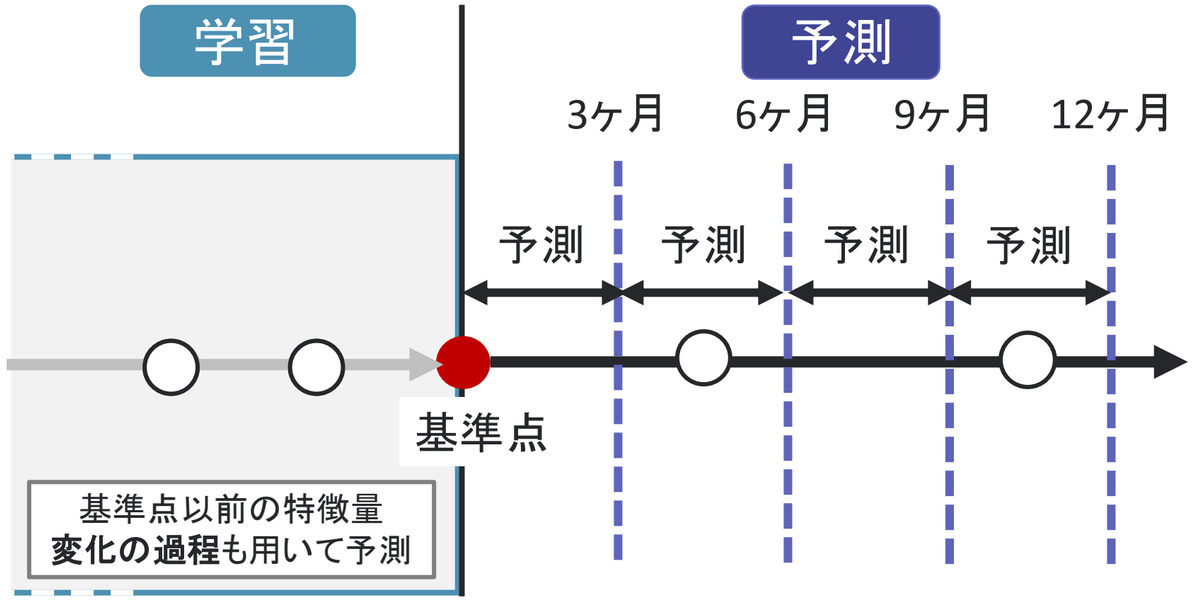
- 特定期間以降の貢献の有無を予測



開発者の一時的な離脱を予測できない

本研究

- 将来の貢献を区間で分割し予測



開発者の離脱・復帰などの時系列パターンも予測

キーアイデア：逆強化学習

開発者を「レビュー依頼に対して 承諾／拒否 を選ぶ主体」として捉える

1 時系列で学習する
累積統計量では承諾パターンの変化が消える

2 強化学習の枠組みで捉える
状態 s＝レビュー履歴、行動 a＝承諾／拒否

3 報酬関数が未知→逆強化学習
行動履歴から逆算し、承諾する確率を出力

累積統計量

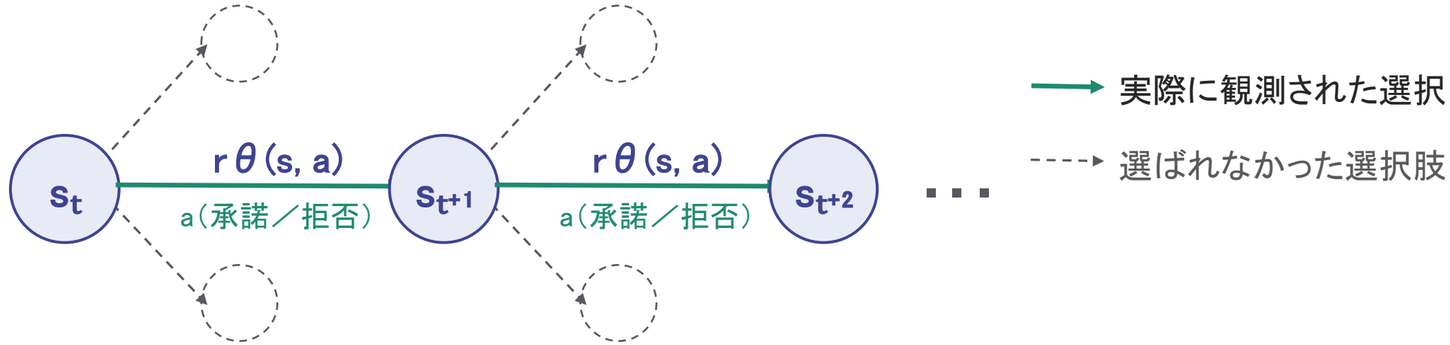


月次の承諾（1）／非承諾（0）

承諾率 = 0.50

いつ止まったかは平均に埋もれる

状態遷移として学習（IRL）



各時点で他の選択肢もあった中で、実際にどれを選んだかを軌跡として学習する

他の選択肢もあった中である選択が繰り返された ⇒ 判断基準 $r_\theta(s, a)$ を逆算できる

従来研究との予測粒度の違い

モジュール単位でどの開発者が継続するかを予測する

従来：開発者単位

この開発者は将来も活動しているか？

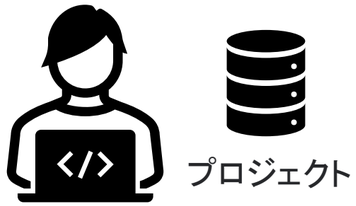
開発者が残っていても，そのモジュールをレビューする人がいなければ，保守は困難

本研究：開発者 × ディレクトリ

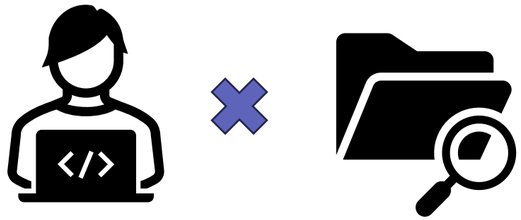
このディレクトリの依頼に，将来この開発者は貢献するか？

モジュール単位で貢献者を確保できるかを見積もることが可能

novaプロジェクトに貢献するか



novaプロジェクトのnova/computeに貢献するか



例:
- nova/compute
- nova/API
- neutron/agent

目的

特定の開発者×ディレクトリのペアにおいて，将来レビュー依頼に対応するか否かを連続的に予測

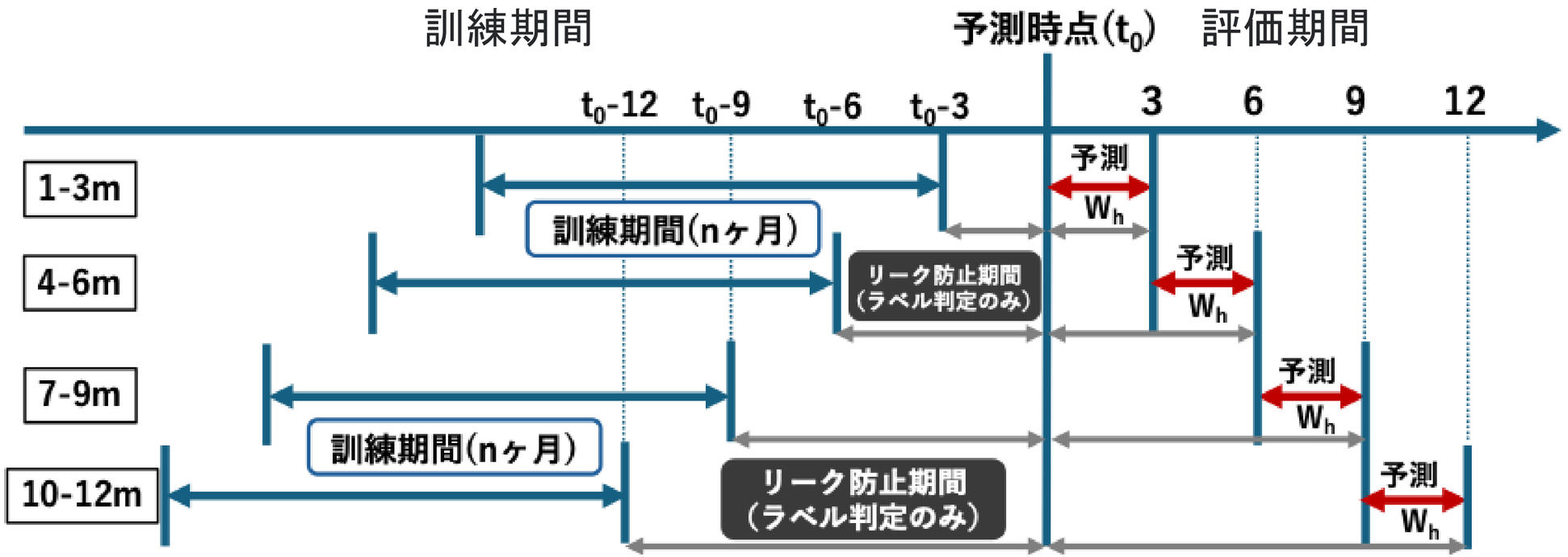
アプローチ概要

1. 逆強化学習モデルの構築，比較手法との比較
2. 比較手法との補完性の検証

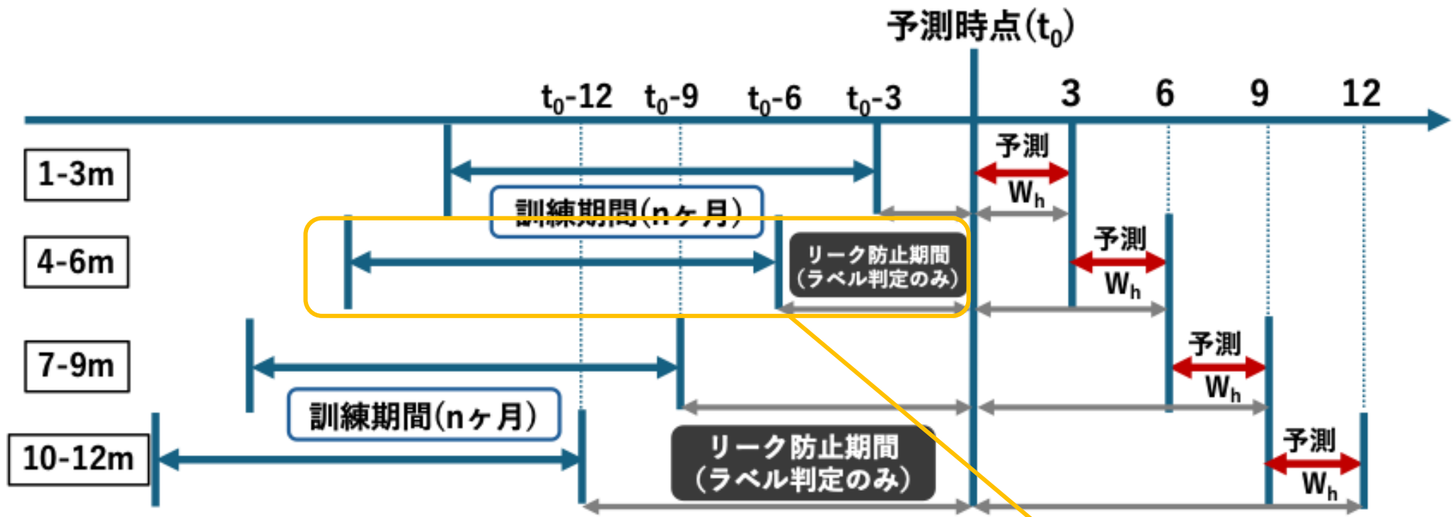
アプローチ（逆強化学習モデルの構築，比較手法との比較）
モデルの全体像

開発者×ディレクトリのペアで各期間の貢献を予測する4つのモデルを作成

- 予測時点(t0)以降の期間を3ヶ月ずつ（W_1:1−3m/W_2:4−6m/W_3:7−9m/W_4:10−12m）予測



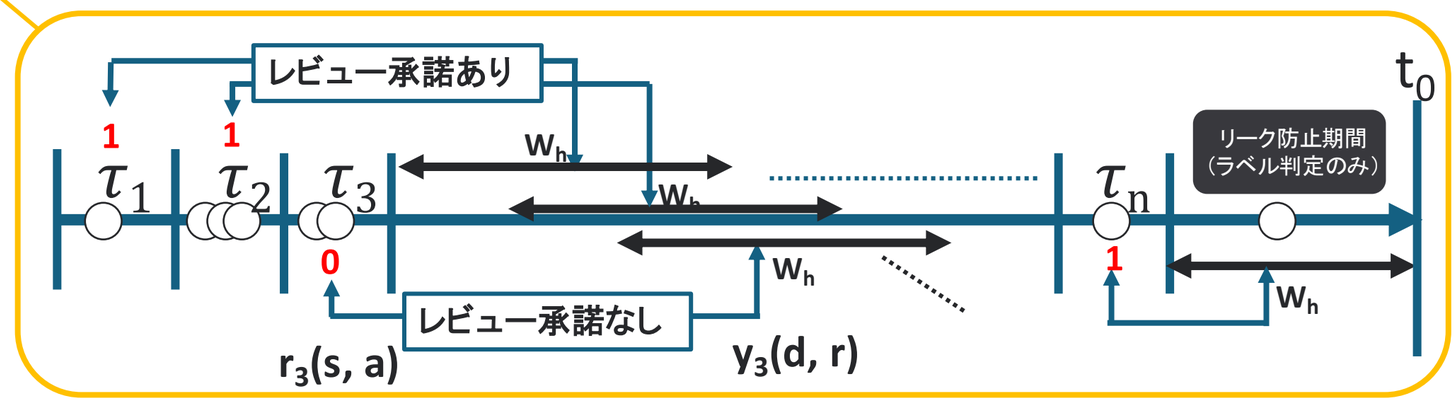
アプローチ（逆強化学習モデルの構築，比較手法との比較）
訓練期間の詳細な学習方法



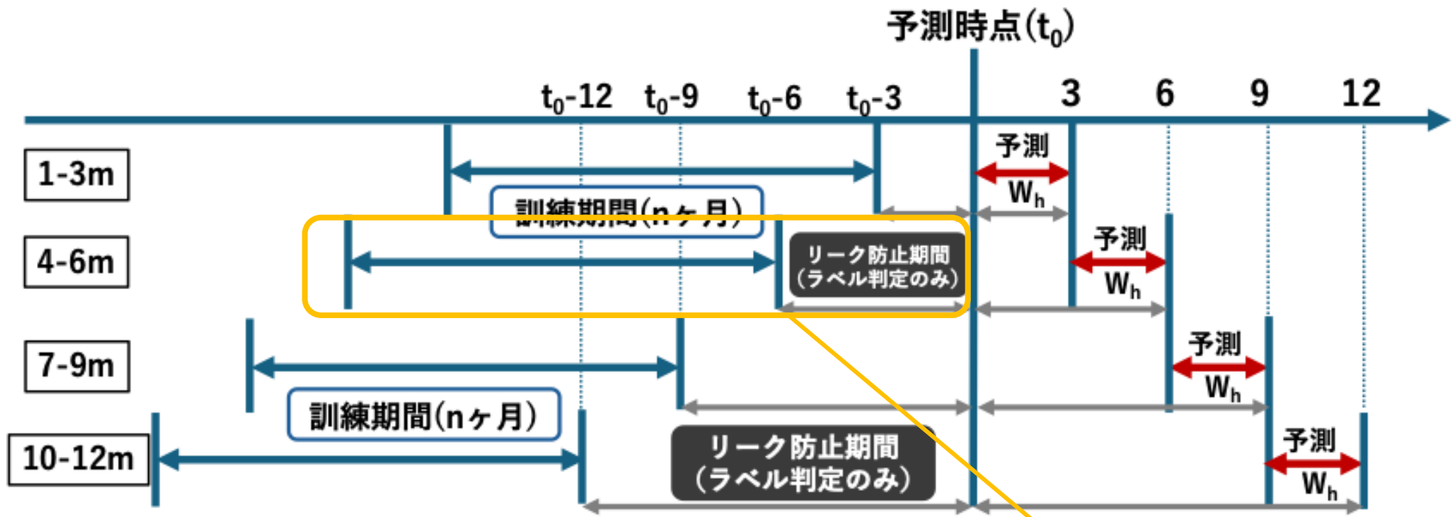
ラベル$y_t(d, r)$：各月τ_tにW_h内の貢献で付与

+1	依頼あり かつ 1件以上貢献 ＝ 正例（重み 1.0）
0	依頼あり かつ 貢献なし ＝ 強い負例（重み 1.0）
0*	依頼なし ＝ 弱負例（重み 0.5）

- **状態 s_t**：
  開発者のそれまでのレビュー履歴(月次集約)
- **軌跡 τ_t**：
  状態+ラベルの月次記録。$T = (\tau_1, \ldots, \tau_n)$ として学習
- **報酬 $r_t(s, a)$**：
  依頼を受けるかの判断基準．IRLの推定対象



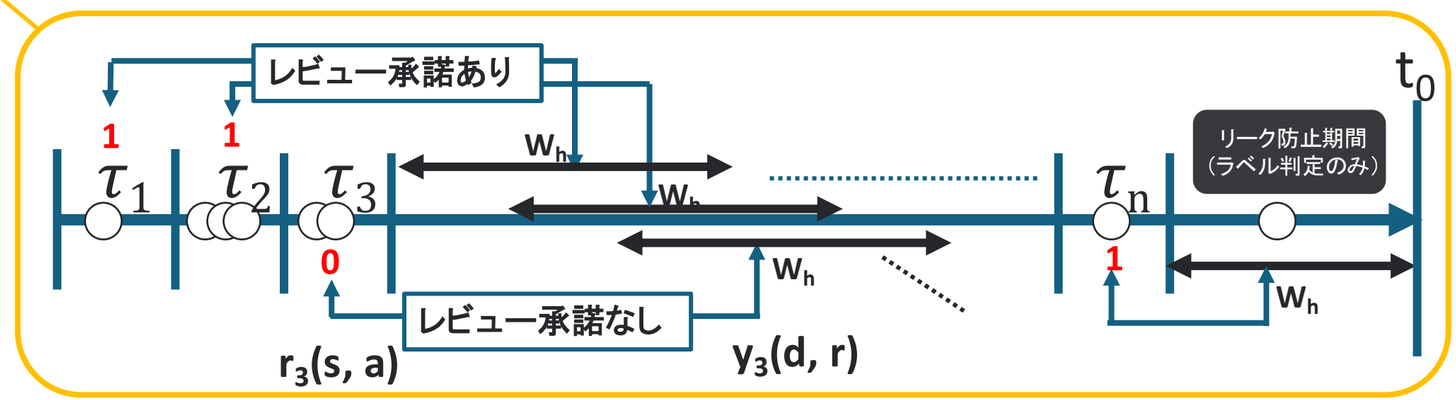
アプローチ（逆強化学習モデルの構築，比較手法との比較）
訓練期間の詳細な学習方法



ラベル$y_t(d, r)$：各月τ_tにW_h内の貢献で付与

+1	依頼あり かつ 1件以上貢献 ＝ 正例（重み 1.0）
0	依頼あり かつ 貢献なし ＝ 強い負例（重み 1.0）
0*	依頼なし ＝ 弱負例（重み 0.5）

- **状態 s_t**：
  開発者のそれまでのレビュー履歴(月次集約)
- **軌跡 τ_t**：
  状態+ラベルの月次記録。$T = (\tau_1, \ldots, \tau_n)$ として学習
- **報酬 $r_t(s, a)$**：
  依頼を受けるかの判断基準．IRLの推定対象



累積統計量では平均化される「活動の停止と再開」，「関与するディレクトリの移り変わり」等を時間経過順に学習可能

アプローチ（逆強化学習モデルの構築，比較手法との比較）
開発者の状態s_tを表現する特徴量

3つのカテゴリ計37次元の特徴量を用いる

カテゴリ	次元数	特徴量（抜粋）
開発者状態	**26**	累積レビュー数・受諾率・直近性・レビュー承諾／非承諾の変動回数 等
ディレクトリ親和度	**6**	過去レビュー数・直近性・受諾率・所有者重なり 等
活動特徴	**5**	行動強度・変更行数・レビュー規模・応答時間・協働の反復率

- 従来研究では，評価対象プロジェクト内の貢献のみを特徴量とすることが多い
- 本研究では，他のプロジェクトへの活動も計測対象に含めることで，
  「他プロジェクトでの多忙により一時的に不活発」「開発活動そのものを停止」といった，
  プロジェクト内の履歴のみでは区別できない状況の把握を可能にする

ケーススタディ

対象プロジェクトの選定

- OpenStack:
  - 公式サービスリポジトリ33件のうち，レビュー履歴のないsunbeamを除く32件のリポジトリ
- AOSP:
  - 活動量に基づきOpenStackと同様に32リポジトリを選定

訓練・評価期間

- 訓練期間3年，評価期間1年，t_0(予測開始地点)2023年1月１日

	データセット全体				予測対象（訓練・評価の両期間で観測）	
プロジェクト	リポジトリ	チケット	レビュー依頼行	開発者	開発者	ペア
OpenStack	32	42,441	161,111	2,372	**436**	**4,534**
AOSP	32	408,574	574,810	8,901	**2,508**	**26,830**

評価方法と比較手法

評価方法

- AUC-ROC: 貢献する開発者としない開発者をどれだけ正しく識別できるか
- 有意性の判定：bootstrap法による95%信頼区間

比較手法：Random Forest

- 従来手法で広く用いられる．決定木の多数決により貢献確率を算出
  - 学習設定は既定値：木の数100，深さ無制限，葉あたり最小サンプル1，乱数シード777，class_weight=balanced（探索なし）

アブレーション：LSTM

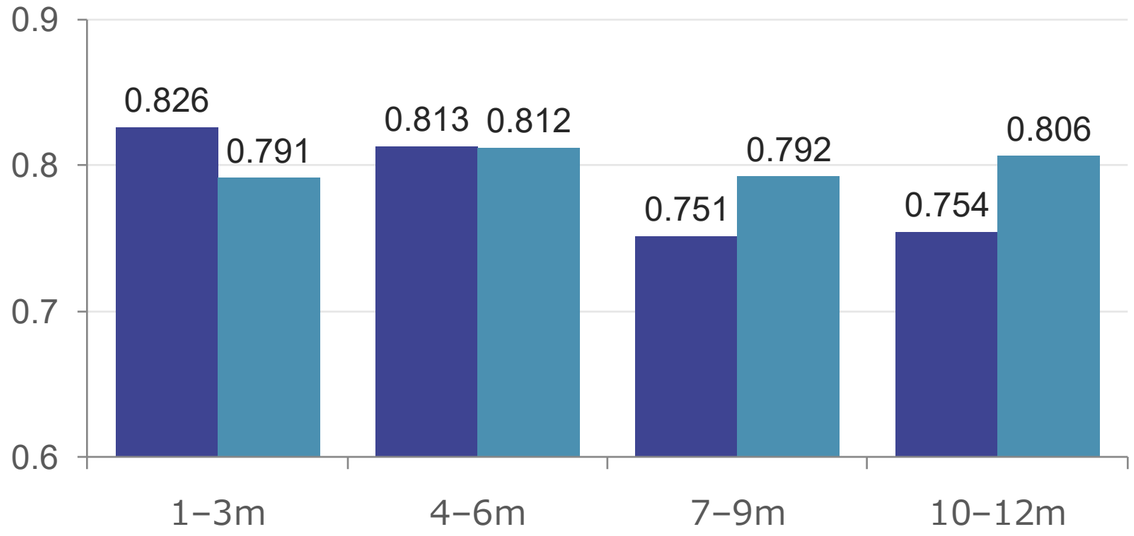
- 「時系列を学習したこと」，「報酬関数として定式化したこと」のどちらが有効であったのかを検証するために比較

結果

IRLとRFは同程度の予測精度

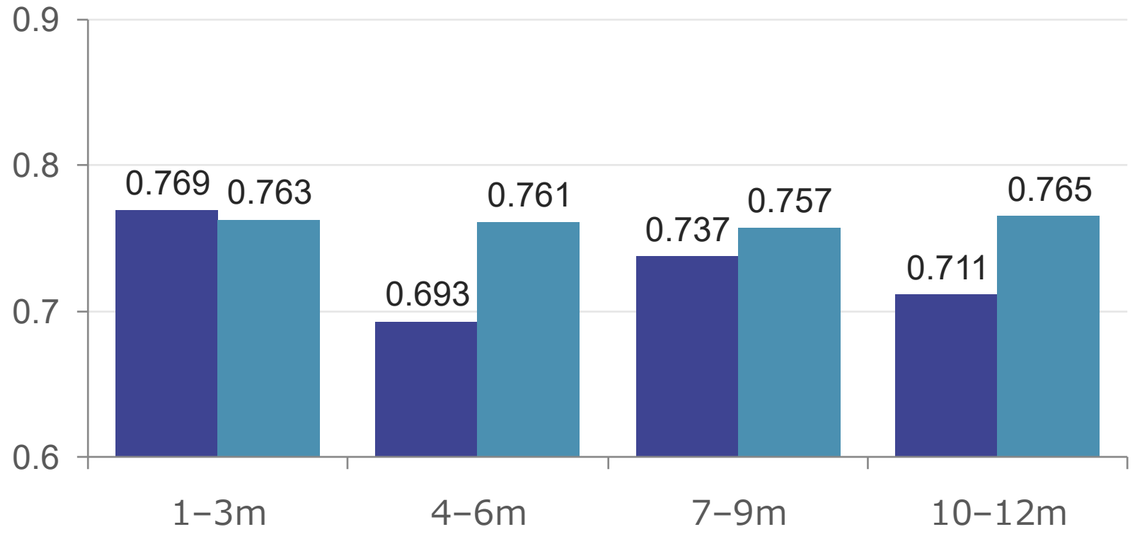
- 4期間のAUCはOpenStack, AOSPともにRFが上回った
- 短期(1–3m)ではIRLがわずかに優位な傾向が見られた

OpenStack（ペア単位、n = 4,534）



Δ **+3.5** / **+0.1** / **−4.1** / **−5.2**　4期間平均 **−1.4**

AOSP（ペア単位、n = 26,830）



Δ **+0.6** / **−6.8** / **−2.0** / **−5.4**　4期間平均 **−3.4**

結果：時系列や報酬関数の定式化のアブレーション

4つの評価期間を統合し．IRL, LSTM, RFの3つの精度を比較

モデル	OpenStack AUC	IRL – 比較手法	AOSP AUC	IRL – 比較手法
IRL（提案手法）	**0.789**	—	**0.727**	—
LSTM（報酬関数の定式化なし）	0.751	**+3.8 pp**	0.764	**−3.7 pp**
RF（累積統計量）	0.801	−1.2 pp	0.776	−4.9 pp

- IRL, LSTMともにRFの精度を下回る → 時系列モデルだけでは不十分
- OpenStackではIRLがLSTMを上回る → 報酬関数の定式化が寄与
  - しかし，AOSPでは逆転しており，プロジェクトの規模や性質により左右されることが示唆

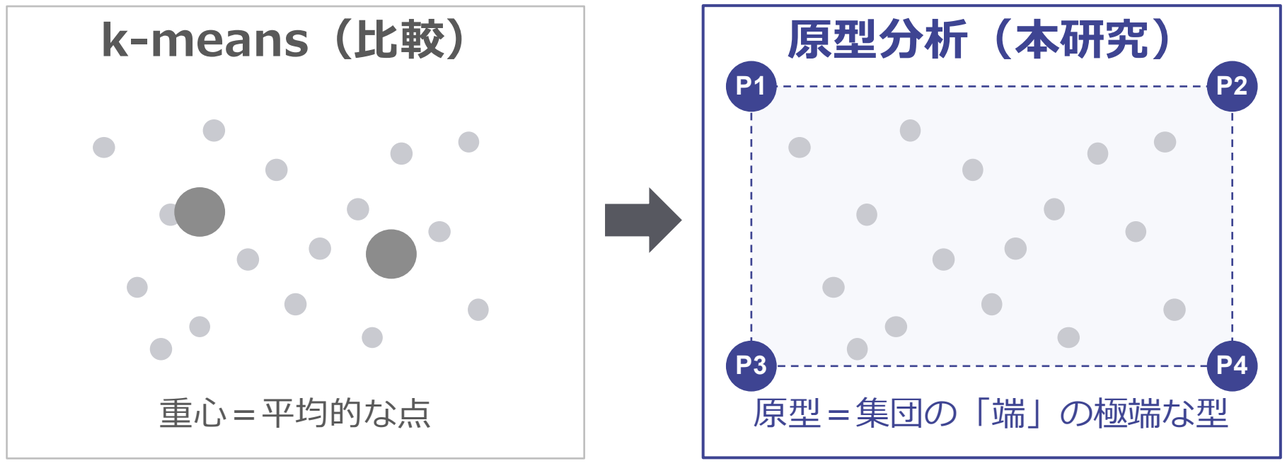
時系列学習，報酬関数の定式化により変化したことがないのかを検証したい

アプローチ：比較手法との補完性の検証

原型分析[1]

極端なパターンを K 個抽出し、各データ点をその重み付き和で近似する手法

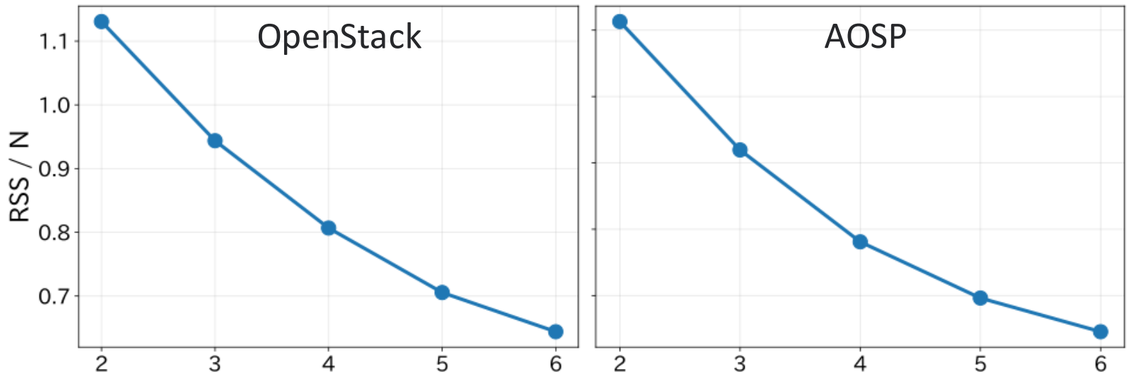
■ K-meansとの違い



中心が平均に近く，クラスタ間での特徴に差が出にくい

原型は特徴量の偏りが大きくタイプごとの特徴を分離しやすい

■ 原型数Kの決め方



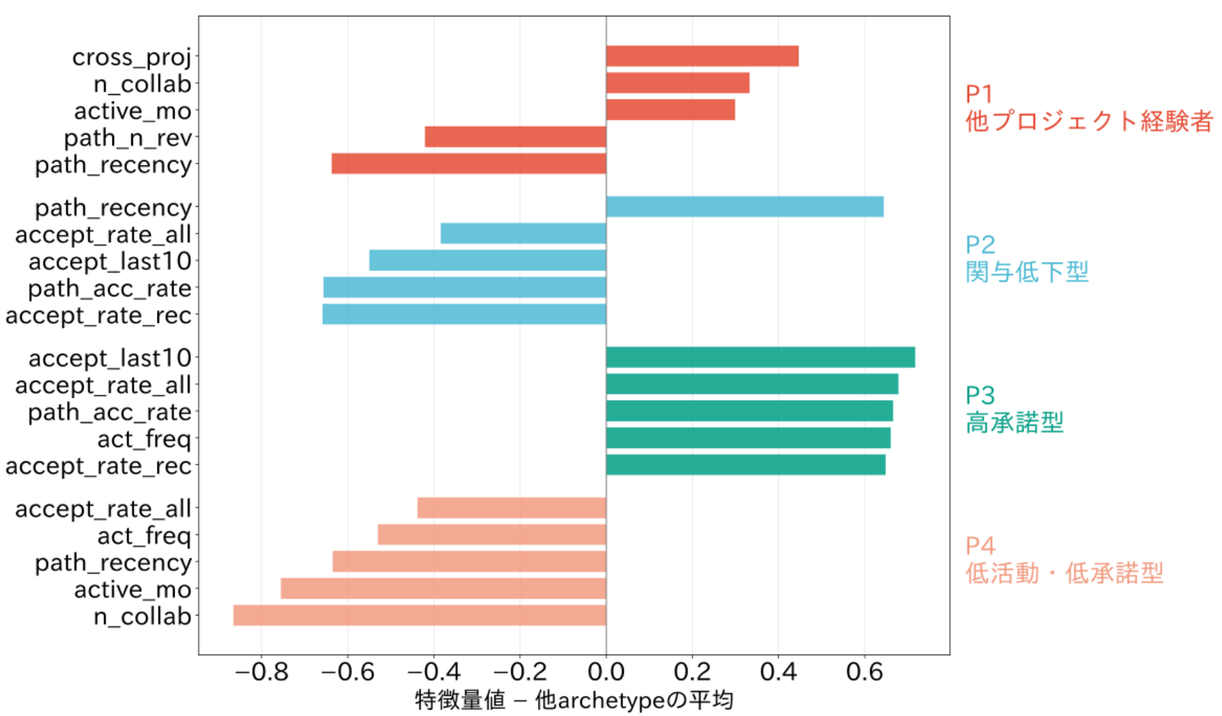
図：原型数 K に対する平均残差平方和の推移

- RSS /Nの改善が小さくなる点（エルボー）で決定
- K=3→4の改善が大きく，5以降は小さいため両プロジェクトでK=4

分析対象：t0 直前6か月に履歴がある開発者（OpenStack 289 開発者・3,747 ペア，AOSP 1,248 開発者・21,291 ペア）

[1] Cutler, A. and Breiman, L.: Archetypal analysis, Technometrics, Vol. 36, No. 4, pp. 338–347 (1994).

結果：開発者タイプごとに、当たるモデルが異なる（OpenStack・ペア単位）



評価期間別 Δ ＝ IRL − RF（pp）

	1–3m	4–6m	7–9m	10–12m	ΣΔ
P1	+9.0	+3.9	+4.2	−5.4	**+11.7**
P2	+2.0	−1.5	−0.1	−4.2	**−3.8**
P3	+5.8	−2.3	−7.0	−0.7	**−4.2**
P4	−0.4	−0.5	−8.7	−5.9	**−15.5**

ΣΔ ＝ IRL − RF の 4 期間合計（pp）

他プロジェクト経験者（n=1,284）

P1　ΣΔ **+11.7**

- 他プロジェクト貢献(cross_proj)が多く，対象ディレクトリの実績(path_n_rev・path_recency)は乏しい
- 4期間中3期間で IRL が優位 ⇒ 時系列情報が効く

関与低下型（n=928）

P2　ΣΔ **−3.8**

- 対象ディレクトリの最終活動からの経過時間(path_recency)が長く，承諾率は低め
- 短期のみ IRL 優位，中長期は RF が優位

高承諾型（n=539）

P3　ΣΔ **−4.2**

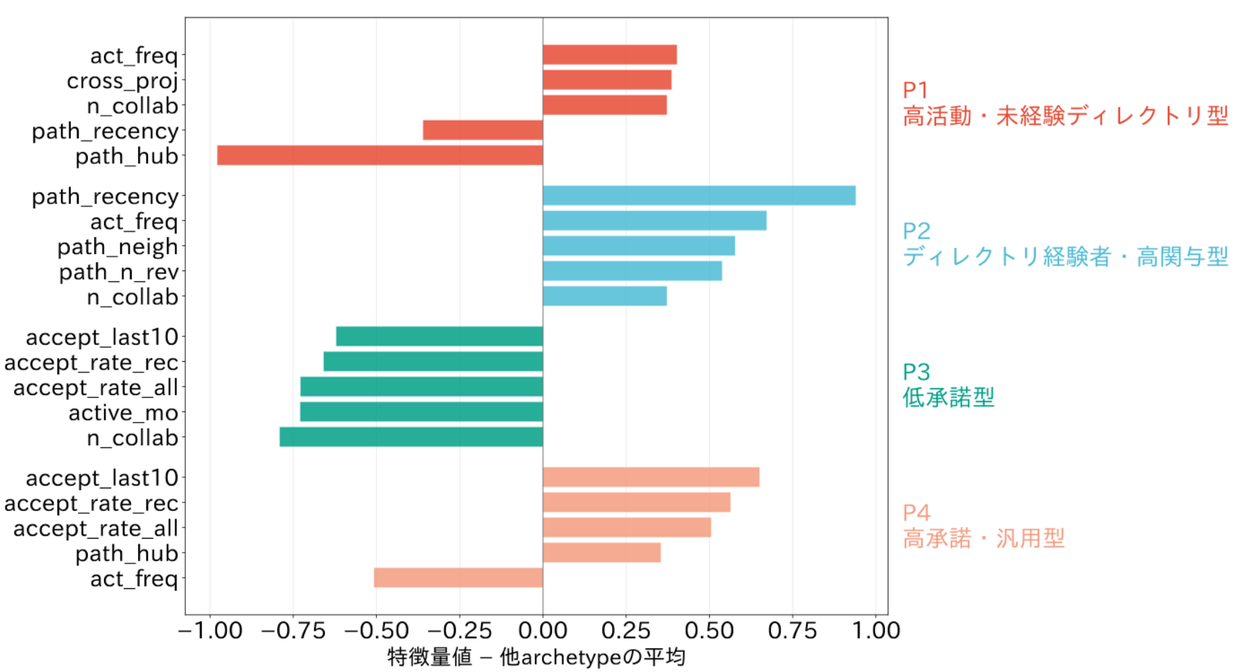
- 承諾率（accept_last10・accept_rate_all 等）が全般に高い
- 累積レビュー実績が豊富 ⇒ RF が有効

低活動・低承諾型（n=926）

P4　ΣΔ **−15.5**

- 活動頻度(act_freq)・承諾率(accept_rate_all)が低い
- 一貫して RF 優位：軌跡が短く IRL の学習が困難

結果：開発者タイプごとに、当たるモデルが異なる（AOSP・ペア単位）



評価期間別 Δ ＝ IRL − RF（pp）

	1–3m	4–6m	7–9m	10–12m	ΣΔ
P1	+0.4	−3.3	−0.2	−8.6	−11.6
P2	−4.4	−11.4	−9.8	−12.4	−38.0
P3	+1.5	−0.5	−2.9	−2.6	−4.6
P4	+4.6	+2.3	+3.9	+3.5	+14.3

ΣΔ ＝ IRL − RF の 4 期間合計（pp）

高活動・未経験ディレクトリ型（n=5,068）

P1　ΣΔ −11.6

- 活動頻度(act_freq)・他プロジェクト経験(cross_proj)は高いが、対象ディレクトリの直近活動(path_recency)は低い
- 長期ほど RF が優位(10–12mで−8.6)

ディレクトリ経験者・高関与型（n=4,422）

P2　ΣΔ −38.0

- 対象ディレクトリの直近活動・過去レビュー(path_n_rev)に加え、活動頻度・協力者数も高い
- 一貫して RF が優位：累積実績が厚いタイプで RF が有効

低承諾型（n=2,878）

P3　ΣΔ −4.6

- 承諾率(accept_last10・accept_rate_all)・活動月数・協力者数がいずれも低い
- 短期のみ IRL 優位、中長期は RF

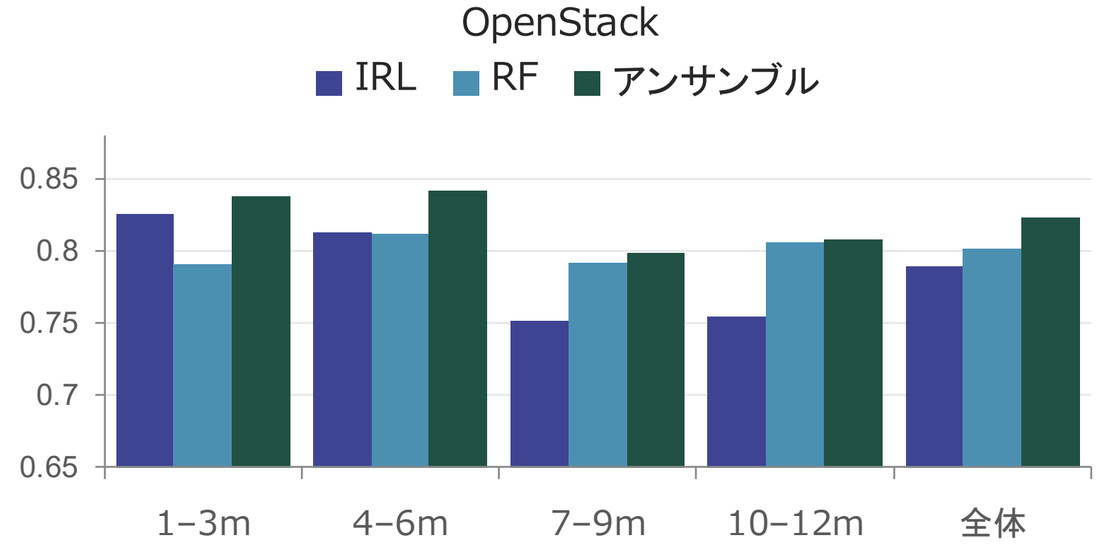
高承諾・汎用型（n=8,923）

P4　ΣΔ +14.3

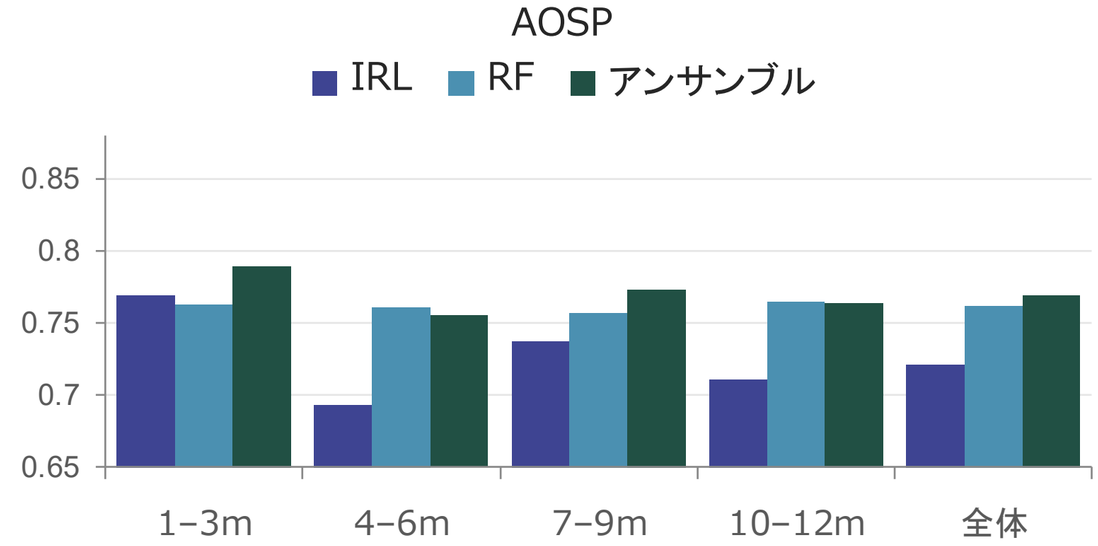
- 承諾率は高い一方、活動頻度(act_freq)は低い
- 全期間で IRL が優位

追加実験：アンサンブル
IRLとRFが出力する確率を単純平均し予測値とする

互いの誤りが相殺され精度が向上することが期待される



	1–3m	4–6m	7–9m	10–12m	全体
改善幅	**+1.2 ✓**	**+2.9 ✓**	+0.6	+0.2	**+2.3 ✓**



	1–3m	4–6m	7–9m	10–12m	全体
改善幅	**+2.0 ✓**	**−0.6 ✓**	**+1.6 ✓**	−0.1	**+0.7 ✓**

最良モデルからの改善幅（pp）。✓ = bootstrap 95% CI が 0 を含まない（有意）

全体：OpenStack +2.3 [+1.7, +2.8]／AOSP +0.7 [+0.5, +1.0]

- 全体ではOpenStack(+2.3), AOSP(+0.7)と両エコシステムで改善が見られた
- AOSPの4-6は有意に悪化が見られた→一方が大きく劣る期間では単純平均が逆効果になる可能性が示唆

考察

軌跡が短い開発者

- IRLは行動パターンをうまく学習できないが，RFは累積の活動量が少ないこと自体が活動停止の予測に寄与する

承諾パターンが変化する開発者

- RFは訓練期間の低い承諾率を重視するが，IRLは行動軌跡から状態の変化を活用することができる

予測期間の長さ

- 長期になるほど，一時的な行動パターンの変化が平均化され，累積特徴量を用いるRFが安定

エコシステムの多様性

- 規模が大きいほど1つのディレクトリに多様な開発者が関与し，行動軌跡から一貫した判断基準を推定しにくくなる

まとめ・今後の展望

まとめ

- 開発者×ディレクトリのペアに対して，レビュー貢献を連続的に予測するIRLモデルを構築
- RFとIRLを比較したところ予測精度は同程度であった
  - 原型分析により，予測が得意な開発者のタイプが異なる可能性を示し，両モデルの予測確率を単純平均するアンサンブルモデルにより精度が向上した

今後の展望

- 推定した報酬関数を環境のフィードバックとして組み込み，開発者の継続確率を考慮したレビュー依頼の推薦方策の設計・評価
- 原型や評価期間に応じてIRL/RFを選択的に採用することによる精度改善

補足資料